ФГБОУ ВО «Пензенский государственный университет»
Факультет вычислительной техники
Кафедра «Вычислительная техника»

ОТЧЕТ

по дисциплине «Логика и ОА»
лабораторная работа №8 на тему:
«Обход графа в ширину»

Выполнил студенты группы 25ВВВ3:
Кустов И. С
Никитин К. А
Приняли:
к.т.н., доцент Юрова О. В.
к.т.н., доцент Деев М. В.

Пенза 2026

Тема: «Обход графа в ширину»

Цель работы: Изучить алгоритм обхода графа в ширину (Breadth-First Search, BFS). Освоить реализацию BFS для графов, представленных в виде матрицы смежности и списков смежности. Изучить структуру данных «Очередь» (FIFO) и получить практические навыки её реализации как стандартными средствами (std::queue), так и собственными силами (на основе односвязного списка). Провести сравнительный анализ времени выполнения алгоритма с использованием встроенной и собственной очереди. Закрепить навыки работы с динамической памятью и измерения времени работы программ.

Лабораторное задание:

Задание 1:

1. Сгенерируйте (используя генератор случайных чисел) матрицу смежности для неориентированного графа G. Выведите матрицу на экран.

2. Для сгенерированного графа осуществите процедуру обхода в ширину, реализованную в соответствии с приведенным выше описанием. При реализации алгоритма в качестве очереди используйте класс queue из стандартной библиотеки C++.

3. Реализуйте процедуру обхода в ширину для графа, представленного списками смежности.

Задание 2:

1. Для матричной формы представления графов реализуйте алгоритм обхода в ширину с использованием очереди, построенной на основе структуры данных «список», самостоятельно созданной в лабораторной работе № 3.

2. Оцените время работы двух реализаций алгоритмов обхода в ширину (использующего стандартный класс queue и использующего очередь, реализованную самостоятельно) для графов разных порядков.

Пояснительный текст:

В отличие от обхода в глубину (DFS), который идет по одной ветке до тупика, обход в ширину (BFS, Breadth-First Search) исследует граф «волнами» или «слоями». Сначала посещается стартовая вершина, затем все её соседи (1-й уровень), затем соседи соседей (2-й уровень) и так далее.

Для реализации такого порядка используется структура данных Очередь (Queue), работающая по принципу FIFO (First In, First Out — «первым пришел, первым ушел»). Это как очередь в магазине: кто встал раньше, того и обслужат раньше.

Во всех алгоритмах используется массив NUM, где 0 означает, что вершина еще не посещена, а 1 — что посещена.

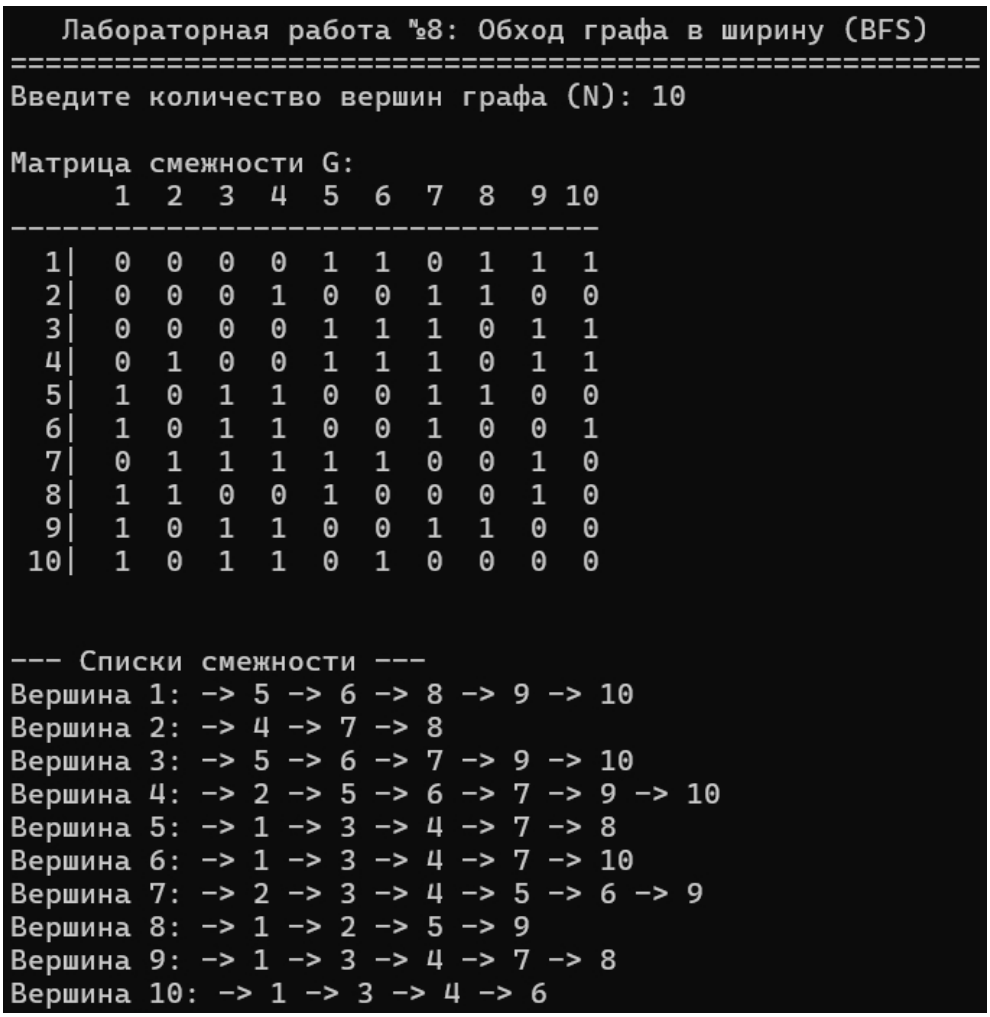
Задание 1.

1. Сгенерируйте матрицу смежности для неориентированного графа G. Выведите матрицу на экран.

Как реализовано в коде:

Этот пункт выполняется автоматически при запуске программы (а также по пунктам меню 6 и 7). За это отвечают функции generate_graph, print_matrix и matrixToAdjList.

- Генерация (generate_graph): Программа создает двумерный массив (матрицу). Главная диагональ заполняется нулями (петель нет). Верхний треугольник матрицы заполняется случайными числами 0 или 1 (rand() % 2). Нижний треугольник дублирует верхний, чтобы граф был неориентированным (если 1 связана с 2, то и 2 связана с 1).
- Вывод: Программа выводит матрицу в красивом табличном виде, а также сразу выводит списки смежности.



2. Для сгенерированного графа осуществите процедуру обхода в ширину с использованием класса queue из стандартной библиотеки C++.

Пункт меню в коде: 1

Функция: BFS_matrix_std

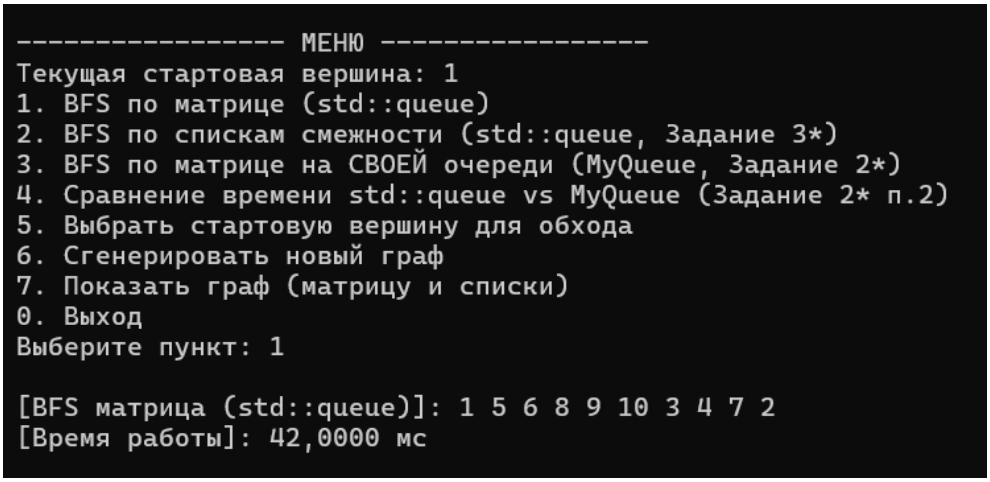
Как работает:

- Создается стандартная очередь std::queue<int> q.
- Стартовая вершина кладется в очередь и помечается как посещенная.
- Запускается цикл, пока очередь не пуста:
  - Достаем вершину v из начала очереди.
  - Смотрим её строку в матрице смежности. Если находим соседа i, который еще не посещен (NUM[i] == 0), мы кладем его в конец очереди и ставим пометку.

Пример:

- Граф: 1 соединена с 2 и 3. 2 соединена с 4.

- Очередь: [1]. Посетили 1.
- Достаем 1. Ее соседи 2 и 3 не посещены. Кладем их в очередь.
- Очередь: [2, 3]. Достаем 2. Ее сосед 4 не посещен. Кладем 4 в очередь.
- Очередь: [3, 4]. Достаем 3. Соседей нет.
- Очередь: [4]. Достаем 4.



3. Реализуйте процедуру обхода в ширину для графа, представленного списками смежности.

Пункт меню в коде: 2

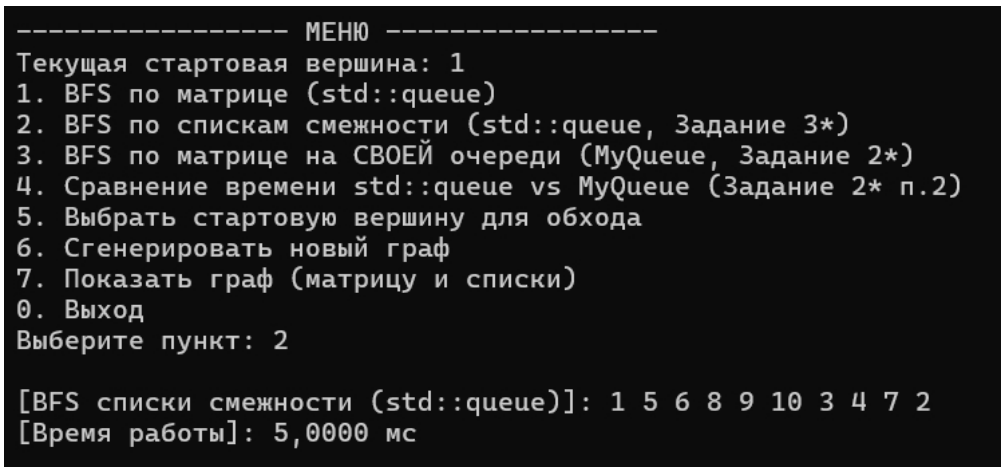
Функция: BFS_list_std

Как работает:

Алгоритм абсолютно идентичен предыдущему, но вместо того, чтобы перебирать всю строку матрицы (в том числе пустые нули), мы идем строго по связному списку соседей текущей вершины v.

```
Node* curr = adjList[v];
while (curr != NULL) {
   int neighbor = curr->vertex;
   if (NUM[neighbor] == 0) { /* ... */ }
   curr = curr->next;
}
```

В матрице мы всегда делаем N шагов для каждой вершины, даже если у нее 1 сосед. В списках мы делаем столько шагов, сколько реально существует ребер. Для разреженных графов (где мало связей) это работает в разы быстрее.



Задание 2.

1. Для матричной формы представления графов реализуйте алгоритм обхода в ширину с использованием очереди, построенной на основе структуры данных «список», самостоятельно созданной в лабораторной работе № 3.

Пункт меню в коде: 3

Функция: BFS_matrix_myqueue

Как работает:

Здесь мы не используем std::queue. Мы написали свою очередь на основе односвязного списка.

- В структуре MyQueue есть два указателя: front (начало очереди, откуда забираем) и back (конец очереди, куда добавляем).
- pushQueue: создает новый узел через malloc и цепляет его в конец (после back).
- popQueue: берет значение из front, сдвигает front на следующий элемент и удаляет старый узел через free.

Логика самого обхода BFS при этом не меняется — мы просто заменили готовую коробку на свою.

```
------------------ МЕНЮ ------------------
Текущая стартовая вершина: 1
1. BFS по матрице (std::queue)
2. BFS по спискам смежности (std::queue, Задание 3*)
3. BFS по матрице на СВОЕЙ очереди (MyQueue, Задание 2*)
4. Сравнение времени std::queue vs MyQueue (Задание 2* п.2)
5. Выбрать стартовую вершину для обхода
6. Сгенерировать новый граф
7. Показать граф (матрицу и списки)
0. Выход
Выберите пункт: 3

[BFS матрица (MyQueue)]: 1 5 6 8 9 10 3 4 7 2
[Время работы]: 3,0000 мс
```

2. Оцените время работы двух реализаций алгоритмов обхода в ширину для графов разных порядков.

Пункт меню в коде: 4

Как работает:

Программа просит ввести размер графа (например, 500 или 1000 вершин). Затем она создает огромный случайный граф и запускает BFS дважды:

- С использованием стандартного std::queue.
- С использованием собственной MyQueue.

С помощью функции clock() замеряется время выполнения.

Ожидаемый результат: Стандартная очередь почти всегда работает быстрее. Это связано с тем, что std::queue оптимизирована на уровне процессора и лучше использует кэш-память. Наша MyQueue при каждом добавлении элемента вызывает malloc (выделение памяти), что является медленной операцией, а элементы в памяти лежат хаотично.

```
---------------- МЕНЮ ----------------
Текущая стартовая вершина: 1
1. BFS по матрице (std::queue)
2. BFS по спискам смежности (std::queue, Задание 3*)
3. BFS по матрице на СВОЕЙ очереди (MyQueue, Задание 2*)
4. Сравнение времени std::queue vs MyQueue (Задание 2* п.2)
5. Выбрать стартовую вершину для обхода
6. Сгенерировать новый граф
7. Показать граф (матрицу и списки)
0. Выход
Выберите пункт: 4

=== Сравнение времени работы (std::queue vs MyQueue) ===
Введите размер графа для теста производительности (рекомендуется 500-1000): 700
Генерация большого тестового графа (700x700)...
Результаты на графе N = 700:
1. Стандартный std::queue : 10,000 мс
2. Собственная MyQueue    : 11,000 мс
=======================================================
```

Вывод:

В ходе выполнения лабораторной работы №8 был изучен и успешно реализован алгоритм обхода графа в ширину (Breadth-First Search). Были рассмотрены две основные формы представления графа: матрица смежности и списки смежности.

Экспериментальное сравнение времени работы показало наглядные различия в эффективности:

1. Матрица vs Списки смежности: Обход по спискам смежности работает быстрее и требует меньше памяти на разреженных графах, так как алгоритм обращается только к реально существующим рёбрам, а не перебирает нули в матрице. Сложность BFS со списками составляет O(V+E), в то время как с матрицей — $O(V^2)$.

2. std::queue vs MyQueue: Обход с использованием стандартной очереди выполняется быстрее. Это обусловлено тем, что std::queue оптимизирована на низком уровне, а самописная очередь при каждом добавлении элемента вызывает функцию динамического выделения памяти malloc, что создаёт дополнительные накладные расходы и снижает локальность кэша процессора.